COM 14104 - Sistemas Operativos

semestre enero – mayo 2026
Primer Examen Parcial
martes 10 de marzo de 2026
duración 1:45

CU: **188653** Nombre: **Juan Pablo Cano Rubio**

Se le proporciona un código base en C para Linux para el ejercicio de los números primos. El código incluye los forks() para ejecutar los hijos.

El examen en salón tiene un valor de 7.0 puntos, elija un tema:

[7.0] Ejercicio I) Pipes Linux: Agregarle los pipes para que los hijos comuniquen el resultado al padre.
Ejercicio II) Memoria Compartida Linux.
Ejercicio III) Pipes en Windows.
Ejercicio IV) Memoria Compartida en Windows.

Solamente del Ejercicio I) se proporciona un código base junto con el examen.

El código "padre" debe ejecutar "procesos hijo" para la obtener la cantidad de números primos con parámetros adecuados:

NUM_HIJOS (<= 10)
MODALIDAD_OUT_LOOP (0 implica que espera a que cada hijo termine, 1 los espera al final)

Cada proceso hijo debe recibir (in situ si son forks o por medio de los argumentos en la línea de comandos si son exec o spawn):

IMPRIME (0 no imprime, 1 imprime)
n_inicial para el número inicial del rango de búsqueda de primos (2**n_inicial -1)
n_final para el número final del rango de búsqueda de primos (2**n_final - 1)
quienSoy (>= 0) para que el hijo se identifique

nota: n_inicial y n_final van de 5 en 5:
hijo 0: 5 10
hijo 1: 10 15
hijo 2: 15 20
hijo 3: 20 25 (este ya se tarda, por eso la recomendación de solamente usar 3 hijos).
etc.

El hijo debe responder en el pipe o memoria compartida con una cadena como la siguiente:
(esta cadena debe ser desplegada por "el padre")

quienSoy:0, primos entre(1023,1048575):81853 en 0.066000 segs

Ejercicio Elegido: **Ejercicio I) Pipes Linux: Agregarle los pipes para que los hijos comuniquen el resultado al padre.**

Resto del examen [3.0 puntos]: elaborar el código en C o C++ para las 3 modalidades no realizadas en el salón. Se entrega el domingo 15 de marzo.

Instrucciones:

a) El documento con las evidencias se entrega en Word y en pdf.
b) Trabaje los códigos en una sola carpeta (códigos y documento de evidencias) para que al finalizar su trabajo o terminar el tiempo comprima el directorio y lo suba a CANVAS.
c) Entregará el documento de las evidencias en Word y en pdf, los códigos base y las modificaciones hechas para responder a lo solicitado.

1) Código base. Elija su código base en términos de la modalidad seleccionada. Renombre su código base como base_ejer_(Lin|Win)_(pipes|shmem)_CU.c
   Por ejemplo base_ejer_Lin_pipes_8472.c (8472 es mi CU)

---

pipes

==========

[1.0] Evidencia 1.1) Compile y ejecute el código base. Use como argumentos 3 1 y ejecútelo con la modalidad time si es en Linux.
Reporte la captura de pantalla aquí como la evidencia.

```
juanpa829@JuanPa:/mnt/c/Users/juanp/SistemasOperativos/Respuesta_Examen$ gcc base_primos_fun_fork-1.c primos_fun_sep.
-o examen
juanpa829@JuanPa:/mnt/c/Users/juanp/SistemasOperativos/Respuesta_Examen$ time ./examen 3 1
Computadora: JuanPa
Proceso hijo 0 (PID 984) va a calcular primos.
n_ini:5 ,n_fin: 10
obteniendo los primos entre 31 y 1023
hijo:0, ./examen --- primos entre(31,1023):162 en 0.000020 segs

Esperando por mis hijos...
Proceso hijo 2 (PID 986) va a calcular primos.
n_ini:15 ,n_fin: 20
obteniendo los primos entre 32767 y 1048575
Proceso hijo 1 (PID 985) va a calcular primos.
n_ini:10 ,n_fin: 15
obteniendo los primos entre 1023 y 32767
terminó hijo con p_id 984
hijo:1, ./examen --- primos entre(1023,32767):3340 en 0.001026 segs

terminó hijo con p_id 985
hijo:2, ./examen --- primos entre(32767,1048575):78513 en 0.085554 segs

terminó hijo con p_id 986
-----------------------------------------------
         ./examen
-----------------------------------------------
NUM_HIJOS:          3
MODALIDAD_OUT_LOOP: 1
CPU Time:           0.000497 seg.
Elapsed time:       0.087013 sec.

real    0m0.095s
user    0m0.090s
sys     0m0.000s
juanpa829@JuanPa:/mnt/c/Users/juanp/SistemasOperativos/Respuesta_Examen$
```

[2.0] Evidencia 2) Indique dónde y describa las modificaciones el código para llevar a cabo los pipes (uno por cada hijo). Para los fildes de los pipes use un arreglo de int fd[10][2] (máximo 10 hijos, proteja su código o utilice un malloc considerando que el fd[hijo][0] y fd[hijo[1] deben quedar contiguos).

Creación de (los pipes):

Lo primero que hice fue declarar el arreglo bidimensional "*int fd[10][2]*" antes de empezar el ciclo para que el programa soportara 10 hijos máximo. Después una vez dentro del ciclo y antes ejecutar el fork(), utilice pipe(fd[hijo]) dentro de un if -> "if (pipe(fd[hijo]) == -1)" para hacer más robusto el programa y que se detenga y avise que la tubería se crearó mal.
Lo importante de estas líneas fue la posición antes del fork() porque cuando se clone la memoria, el proceso hijo heredara automáticamente las copias de las llaves de lectura y escritura de su pipe

Mecanismo para que cada hijo se entere de dónde debe responder (para la escritura al pipe por cada hijo):

Como se hizo fork() el hijo ya tiene la memoria del padre. Después para enviar su respuesta hice que el hijo hiciera lo sigueinte:

Primero cierro el extremo de lectura del pipe close(fd[hijo][0]), despues se calcula el resultado y utilico sprintf (en la parte de **TERCERA MODIFICACIÓN**) para darle el formato necesario a la cadena ya que write() no formatea el text y este se manda con el número de hijos y sus primos encontrados, para despues llamara a write() que inyectar los bytes del mensaje a través del extremo de escritura del tubo. Finalmente se cierra su llave de escritura (en la parte de **CUARTA MODIFICACIÓN**) para indicarle al padre que termino la transmisión.

Mecanismo de lectura de cada respuesta (pipe) por el padre: **Escriba aquí.**

La parte del mecanismo del padre consiste en extraer los datos del tubo correspondiente a cada hijo dentro o fuera del ciclo principal según la modalidad:

Primero lo que que hice fue que el padre bloquea su propio acceso de escritura al pipe con "*close(fd[hijo][1])*" (que esta en la parte de **QUINTA MODIFICACIÓN**). Despues asegurar la sincronización y que terminó el hijo, el padre ejecuta la llamada al sistema con "*read(fd[hijo][0], recibe_sms, sizeof(recibe_sms))"* (en la parte de **SEXTA y OCTAVA MODIFICACIÓN**)(usamos qué tubo leer (fd[hijo][0]), guardar el mensaje en *recibe_sms* y límite máximo de bytes *sizeof(recibe_sms)*) para extraer el paquete de texto desde la parte de lectura. Esto despliega la cadena "*recibe_sms*" directamente en la terminal utilizando printf. Por ultimo el proceso de comunicación cierra la llave de lectura restante con "close(fd[hijo][0])" (en la parte de **SÉPTIMA y NOVENA MODIFICACIÓN**).

[3.0] Evidencia 1.3) Lleve a cabo las modificaciones, compile el código y ejecútelo con argumentos 3 1 con la modalidad time.
Reporte la captura de pantalla aquí como la evidencia.

```
sys     0m0.000s
juanpa829@JuanPa:/mnt/c/Users/juanp/SistemasOperativos/Respuesta_Examen$ gcc base_primos_Lin_Pipes_188653-1.c primos_f
n_sep.c -o examen
juanpa829@JuanPa:/mnt/c/Users/juanp/SistemasOperativos/Respuesta_Examen$ time ./examen 3 1
Computadora: JuanPa
Esperando por mis hijos...
terminó hijo con p_id 1065
quienSoy:0, primos entre(31,1023):162 en 0.000022 segs

terminó hijo con p_id 1066
quienSoy:1, primos entre(1023,32767):3340 en 0.001437 segs

terminó hijo con p_id 1067
quienSoy:2, primos entre(32767,1048575):78513 en 0.125401 segs

-----------------------------------------------
         ./examen
-----------------------------------------------
NUM_HIJOS:          3
MODALIDAD_OUT_LOOP: 1
CPU Time:           0.000682 seg.
Elapsed time:       0.125685 sec.

real    0m0.136s
user    0m0.130s
sys     0m0.000s
juanpa829@JuanPa:/mnt/c/Users/juanp/SistemasOperativos/Respuesta_Examen$
```

==========

Memoria Compartida:

==========

[1.0] Utilice como código base el código proporcionado en el previo con la compilación y ejecución sin forks.
Reporte captura de pantalla como Evidencia 2.1)

[2.0] Evidencia 2.2) Indique y describa en qué parte va a agregar la memoria compartida.

Creación de la memoria: **Escriba aquí.**

Mecanismo para que cada hijo se entere de dónde debe responder (para que sepa en dónde escribir en la memoria compartida). Escritura en la memoria por parte de los "hijos": Para la memoria compartida usted debe hacer que cada hijo escriba en el apuntador base de la memoria compartida más 100 caracteres de offset para no encimar las respuestas. Si el apuntador al buffer en la memoria es char *prt el proceso hijo quienSoy debe escribir en prt + 100 * quienSoy
Escriba aquí.

Mecanismo de lectura de cada respuesta (de la memoria compartida) por el padre:

Escriba aquí.

[3.0] Evidencia 2.3) Haga las modificaciones al código, compílelo y ejecútelo con time y los argumentos 3 1.
Reporte captura de pantalla como Evidencia.